Implementation of Backward Chaining

```
RULES = [  # (premises, conclusion)
    (["frog"], "green"),
    (["croaks", "eats_flies"], "frog"),
    (["chirps", "sings"], "canary"),
    (["canary"], "yellow"),
]
FACTS = {"croaks", "eats_flies"}


def backward_chain(goal, facts, rules, depth=0, visiting=None):
    visiting = visiting or set()
    pad = "  " * depth
    if goal in facts:
        print(f"{pad}{goal}: known fact")
        return True
    if goal in visiting:
        return False
    visiting = visiting | {goal}
    for premises, concl in rules:
        if concl == goal:
            print(f"{pad}{goal}: try rule {premises} -> {concl}")
            if all(backward_chain(p, facts, rules, depth + 1, visiting) for p in premises):
                print(f"{pad}{goal}: PROVED")
                return True
    print(f"{pad}{goal}: cannot be proved")
    return False


def program8():
    print("\n=== 8. BACKWARD CHAINING ===")
    print("Facts:", FACTS)
    for goal in ("green", "yellow"):
        print(f"\nGoal: {goal}")
        print("Result:", backward_chain(goal, set(FACTS), RULES))


if __name__ == "__main__":
    program8()
```